

T2IEval: 统一、可扩展的文生图评测框架

从指标复现到 ELLA、GenAI-Bench 与 T2I-CoReBench 的统一接入

徐楷鸣

为什么需要统一的 T2I Evaluation Toolkit ?

现实问题

- 不同 Benchmark 有各自的数据格式与 Prompt 组织。
- 生图参数、评测模型和依赖环境不一致。
- 官方脚本入口、图片目录与结果格式相互割裂。
- 更换 T2I 模型时，往往要重复修改多套代码。



T2IEval 的目标

- 用一个 CLI / YAML 入口运行多个 Benchmark。
- 将生成模型与评测逻辑解耦。
- 统一读取、生成、评分、汇总和结果保存流程。
- 让新增模型或 Benchmark 只实现必要的差异部分。

核心价值：降低重复配置成本，使跨模型、跨 Benchmark 的结果可以复现和对照。

任务范围与汇报顺序

A · 理解 GenEval

跑通参考实现，理解注册、生成、评分与汇总。

B · 接入 ELLA

在框架内实现数据读取、VQA 评分与依赖聚合。

C · 接入 GenAI

接入 CLIP-FlanT5 scorer，复现 SD2.1 / SDXL。

D · 通用接口

完善模型、Benchmark、自定义配置与结果接口。

E · 进阶扩展

接入 T2I-CoReBench，并以本地 Qwen MLLM 评测。

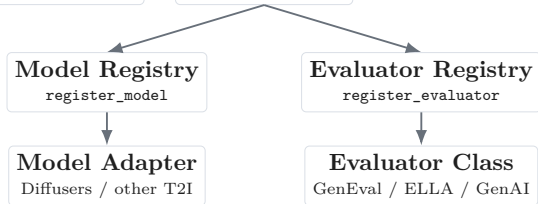
验证目标不是提高 Benchmark 分数，
而是在对齐模型、配置和 scorer 后，复现与论文 / 官方代码接近的指标。

T2IEval 整体代码框架

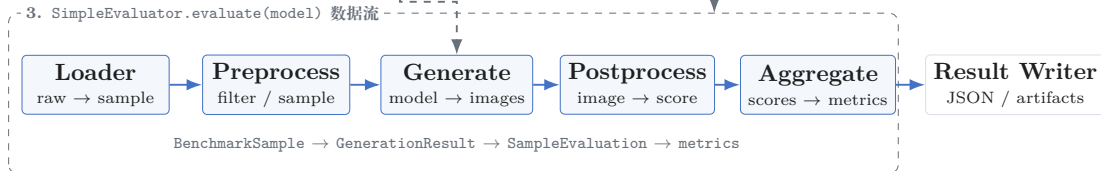
1. 配置与调度



2. 选择实现



3. SimpleEvaluator.evaluate(model) 数据流



核心分工：Runner 只负责调度；Model Adapter 只负责生图；Evaluator 将 Benchmark 特有组件装入统一执行骨架。

任务 A：目录结构、配置入口与数据加载

1. GenEval 目录结构

```
eval/geneval/  
    __init__.py  导入 evaluator，触  
                  发注册  
    evaluator.py  配置类、row 转换、  
                  Postprocessor、  
                  Evaluator 装配  
    utils.py     图像裁剪、CLIP 颜  
                  色分类、IoU、相对位  
                  置  
  
examples/geneval/  
run_geneval_sd15.yaml  
model/diffusers_model.py
```

代码边界：特有评分留在 GenEval 目录；
生图复用公共 Diffusers Model。

2. 配置如何进入 Evaluator

- 1 YAML 提供 SD1.5 路径、50 steps、CFG 9、seed 42、4 images / prompt。
- 2 Runner 合并全局 generation 与任务 override。
- 3 GenevalEvaluator(**kwargs) 构造 GenevalConfig。
- 4 构造函数将 loader、可选 preprocess、Postprocessor 和两个 Aggregator 装入配置。

```
t2i-eval -e geneval  
-f examples/geneval/  
run_geneval_sd15.yaml
```

3. 数据怎样加载和标准化

- 1 load_hf_records 读取 Vertsineu/geneval 的 validation split。
- 2 反序列化 metadata/include/exclude 字段。
- 3 _geneval_row_to_eval_items 将每行转换为 BenchmarkSample。

```
sample_id / prompt  
generation_config  
metadata = {category, prompt,  
            include/exclude rules}
```

全量模式：不做 preprocess；仅设置 category 或 num_samples 时筛选 / 抽样。

任务 A：生成流程、评测流程与结果汇总

4. 生成流程

- ❶ `_generate` 读取样本内的 `GenerationConfig`。
- ❷ `DiffusersModel.generate` 将统一字段映射为 SD Pipeline 参数。
- ❸ 每个 Prompt 生成 4 张图，generator 使用 seed 42-45。
- ❹ 返回 `GenerationResult(images, latency)`。
- ❺ 随后 `model.unload()` 释放 SD1.5 显存，再加载评测模型。

显存切换：先卸载生成模型，再加载 Mask2Former 与 CLIP。

5. 评测流程

- ❶ Mask2Former 预测实例 mask、类别和置信度；普通阈值 0.3，Counting 阈值 0.9。
- ❷ mask 转 bbox，并统一检测器与 GenEval 的类别名称。
- ❸ 对照 metadata 检查 include / exclude 中的对象与数量。
- ❹ 颜色由 OpenCLIP 分类；位置由 bbox 中心与尺寸判断。

每张图输出 `correct + reason`；每个 Prompt 的 `correct` 为 4 张图正确率均值。

6. 结果怎样汇总

$$S_k = \text{mean}_{i:c_i=k}(\text{correct}_i)$$

$$\text{Overall} = \frac{1}{6} \sum_{k=1}^6 S_k$$

- Aggregator 1 输出六类 `task_scores`。
- Aggregator 2 对六类等权平均，输出 `score`。
- 使用 `correct`，不是 `prompt_correct=any(...)`。

`results_geneval_diffusers.json`
`task_scores / score / error`

任务 A：GenEval 指标复现

	Overall	Single object	Two object	Counting	Colors	Position	Attr. binding
论文 SD1.5	0.43	0.97	0.38	0.35	0.76	0.04	0.06
本项目复现	0.4208	0.9938	0.3535	0.3500	0.7500	0.0175	0.0600
相对误差	-2.14%	+2.45%	-6.97%	0.00%	-1.32%	-56.25%	0.00%

复现配置

SD1.5 | 50 steps | CFG 9.0
4 images / prompt | seed 42

结果判断

本项目各项指标与论文报告**基本一致**，完成 GenEval 指标复现。

结果文件: results_geneval_sd15_paper/results_geneval_diffusers.json

任务 B: ELLA 的实现思路与设计取舍

1. 先识别可复用的稳定骨架

- CLI / YAML、Registry 与 Runner 已经解决“如何启动一次评测”。
- `BenchmarkSample` 已经能表达 Prompt、生成参数与 metadata。
- `SimpleEvaluator` 已经固定 load-generate-postprocess-aggregate 生命周期。
- Diffusers Model 已经能够完成 SD1.5 生图和显存释放。

为什么复用：这些能力与 Benchmark 评分语义无关，重复实现只会产生多套入口和结果格式。

2. 再定位 ELLA 的必要差异

- 一个 Prompt 对应多条 Yes/No 问题，而不是一组对象检测规则。
- 每个问题带父子依赖，原始 VQA 正确不代表依赖修正后有效。
- 评分模型是 ModelScope mPLUG，不属于生成模型接口。
- Overall 与论文分类统计使用不同的汇总范围。
- 官方数据还需要 CSV 分组和离线 Parquet 读取。

为什么新增：这些是 ELLA 的评分定义，无法由 GenEval 的检测 Postprocessor 表达。

3. 将差异放到正确的扩展点

- **Loader:** HF / Parquet / CSV → 标准样本。
- **Scorer:** VQAScorer Protocol 隔离 mPLUG，可替换、可测试。
- **Postprocessor:** 逐图问答与 dependency 传播。
- **Aggregator:** Overall、五类与二级类别统计。
- **Policies:** 显式配置缺失依赖和 last-image 行为。

最终结构：公共框架保持不变，只向 eval/ella/ 增加 ELLA 特有实现。

任务 B: ELLA 的框架内实现细节

1. 新增模块及职责

- `__init__.py`: 导出 `EllaEvaluator`, 触发框架注册。
- `loader.py`: HF Dataset 加载与本地 Parquet fallback。
- `evaluator.py`: 配置、CSV 分组、Postprocessor、聚合与注册。
- `scorer.py`: mPLUG adapter 与问题依赖传播。

实现方式: 所有必要代码合入 `eval/ella/`, 不是外部独立脚本。

2. 复用了哪些公共组件

- 执行骨架: `SimpleEvaluator`。
- 标准样本: `BenchmarkSample / SampleEvaluation`。
- 生成与抽样: `DiffusersModel / processor.sample`。
- 运行入口: `Runner`、`Registry`、`CLI / YAML`。
- 结果命名: `results_ella_diffusers.json`。

复用边界: 公共框架负责运行; ELLA 目录只实现数据、VQA、依赖和聚合差异。

3. Evaluator 如何装配

- `@register_evaluator("ella")` 注册统一入口。
- `EllaConfig(**kwargs)` 校验数据、VQA 与聚合参数。
- `loader` 绑定 `load_ella_records`。
- 设置 `num_samples` 时加入随机抽样 `preprocessor`。
- `Postprocessor` 绑定 `EllaPostprocessor`。
- `Aggregator` 绑定 `aggregate_ella_results`。

```
t2i-eval -e ella
-f examples/ella/run_ella_sd15.yaml
```

任务 B：数据读取、Prompt 组织与参数配置

4. 数据读取

- 默认读取 Vertsineu/ella 的 validation split。
- **datasets** 可用时直接读取，并反序列化 **questions**。
- 离线模式失败时，从 HF snapshot 查找 **validation.parquet**，用 **pandas** 读取。
- 也支持 **csv_path**；CSV 按 **item_id** 分组，把多行问题还原为一个样本。

输出：每个数据项转换为一个 **BenchmarkSample**。

5. Prompt 与问题组织

原始 **prompt/text** 直接作为生图 Prompt；同一 **item_id** 下的问题保存在 **metadata**：

- **qid**：问题 / proposition 编号。
- **question**：自然语言 Yes/No 问题。
- **dependency**：父问题编号列表。
- **category**：broad + detailed 两级类别。
- **tuple**：官方语义元组文本。

关键：Prompt 用于生图；questions 用于逐图核查 Prompt 中的细粒度命题。

6. 参数配置入口

- **数据**：dataset / split / csv / subset。
- **生成**：SD1.5, 512², 50 steps, CFG 12, seed 42, 4 images。
- **VQA**：本地 mPLUG COCO large EN。
- **依赖策略**：zero。
- **分类策略**：official_last_image。
- **输出**：output.dir。

覆盖方式：YAML 为默认配置，CLI 可覆盖任务参数与输出路径。

任务 B：生成、评测逻辑与结果保存

7. 生成设置与流程

- Loader 为每个样本复制 `EllaGenerationConfig` 并写入 Prompt。
- 公共 Diffusers Model 使用 SD1.5 生成 512×512 图片。
- 50 steps、CFG 12、seed 42，每个 Prompt 生成 4 张图。
- 生图完成后卸载 SD1.5，再加载 mPLUG VQA，避免同时占用显存。

全量运行：1,065 Prompts $\rightarrow$ 4,260 images。

8. VQA 与依赖评测

- 对每张图逐一询问该 Prompt 的全部问题。
- mPLUG 输出标准化为小写；仅 "yes" 记 1，其余记 0。
- 按问题顺序传播 dependency：父问题为 0，则子问题调整为 0。
- 缺失父问题默认按 zero 处理，同时记录引用错误。
- 保存 raw score、dependency score、valid 和两级类别。

每张图分数 = 依赖修正后问题分数的平均值。

9. 聚合与结果保存

- **Overall**：先平均每张图的依赖修正分数，再平均 4 张图和全部 Prompt。
- **五类指标**：兼容官方 `official_last_image` 行为，取最后一张图的 raw score，按 broad / detailed 类别统计。
- 保存 prompts、images、question evaluations、缺失依赖数和统计策略。

```
results_ella_diffusers.json  
score / task_scores /  
category_scores_12
```

任务 B: ELLA 指标复现

	Overall	Global	Entity	Attribute	Relation	Other
论文 SD1.5	63.18	74.63	74.23	75.39	73.49	67.81
本项目复现	63.68	79.03	72.21	72.93	79.64	52.40
相对误差	+0.79%	+5.90%	-2.72%	-3.26%	+8.37%	-22.73%

复现配置

SD1.5, 512² | 50 steps | CFG 12.0
4 images / prompt | seed 42

结果判断

Overall 与论文报告**基本一致**；分类差异主要来自官方分类聚合策略。

结果文件: results_ella_sd15/results_ella_diffusers.json

任务 C: GenAI-Bench 的实现思路

1. 继续复用稳定骨架

- CLI、YAML、Registry、Runner 不感知具体 Benchmark。
- **BenchmarkSample** 足以保存 Prompt 和 skills 标签。
- Diffusers Model 可通过 YAML 切换 SD2.1 与 SDXL。
- **SimpleEvaluator** 已提供生成、卸载模型、后处理和聚合生命周期。
- Runner 继续负责统一结果命名与错误保存。

为什么复用: 换 Benchmark 不应重新实现模型加载、CLI 和运行管理。

2. 找出 GenAI-Bench 的差异

- 数据不是问题依赖图, 而是 Prompt 加多个 skills 标签。
- 评分不是 Yes/No 正确率, 而是图文对的连续 VQAScore。
- 论文复现指定 `clip-flanT5-xxl` 与 `t2v-metrics==1.1`。
- 同一 Prompt 分数需要同时计入 basic / advanced 和细分 skill。
- Scorer 接口接收图片路径, 需要适配 PIL 图片和批量调用。

为什么新增: 这些数据语义和打分方法不能由 GenEval 或 ELLA 的 Postprocessor 表达。

3. 对应到四个扩展点

- **Loader:** Prompt + skills → 标准样本。
- **Scorer:** PIL 图片 → 临时路径 → CLIP-FlanT5 分数。
- **Postprocessor:** 展平 image-prompt pairs, 批量评分后恢复归属。
- **Aggregator:** 按 skills 分组, 并独立计算 Overall。

设计结果: 新增代码只位于 `eval/genaibench/`, 核心 Runner 与 CLI 无专用分支。

任务 C：框架内代码实现细节

4. 新增模块及职责

- `__init__.py`: 导出 `GenAIBenchEvaluator`。
- `loader.py`: HF Dataset 与本地多 Parquet fallback。
- `scorer.py`: Scorer Protocol、CLIP-FlanT5 adapter 与资源释放。
- `evaluator.py`: 配置、row 转换、Postprocessor、聚合和注册。

所有 Benchmark 特有逻辑都收敛在 `eval/genaibench/`。

5. Scorer Adapter 的处理

- 启动时严格检查 `t2v-metrics==1.1`。
- 只导入所需 `clip_t5_model` 子模块，避免无关可选后端影响加载。
- 将 PIL 图片写入临时 PNG，因为官方 scorer 接收路径。
- 按 `score_batch_size` 调用 `CLIP5Model.forward`。
- Tensor 转为有限 float 列表，完成后释放 scorer 与 CUDA cache。

Adapter 隔离了第三方库接口，Evaluator 只依赖 `score_pairs`。

6. Evaluator 如何装配

- 注册键为 `genaibench`。
- 配置仅接受 `image_1600 / image_527`，并验证 batch size。
- loader 绑定数据集、config、split 与 row converter。
- 可选 `num_samples` 使用公共抽样器。
- Postprocessor 绑定 CLIP-FlanT5 scorer。
- 两个公共 list aggregator 分别输出 `task_scores` 和 `score`。

```
t2i-eval -e genaibench
-f examples/genaibench/
run_genaibench_sd21.yaml
```

任务 C：数据读取、Prompt 组织与参数配置

7. 数据读取

- 数据源：Vertsineu/geneaiibench。
- 论文全量配置：image_1600, split 为 test_v1。
- datasets 读取时反序列化 skills。
- 离线时在 snapshot 中查找 config 对应的全部 split Parquet，并用 pandas 合并。
- 支持 num_samples 构造 smoke subset。

全量运行得到 1,600 个标准样本。

8. Prompt 与 skills 组织

每一行直接转换为一个 BenchmarkSample:

- **sample_id**: prompt_id, 缺失时使用 index。
- **prompt**: 英文 Prompt, 同时写入 GenerationConfig。
- **prompt_zh**: 中文对照, 仅保存在 metadata。
- **skills**: 一个 Prompt 可多个标签, 例如 attribute + basic。

关键：同一个英文 Prompt 同时用于生图和 VQAScore；skills 只决定分数进入哪些类别。

9. 参数配置入口

- **数据**: dataset / config / split / subset。
- **Scorer**: clip-flant5-xxl, v1.1, batch 16。
- **模板**: 可选 question / answer template。
- **SD2.1**: 768², 50 steps, CFG 9, seed 42。
- **SDXL**: 1024², 50 steps, CFG 9, seed 42。
- **输出**: output.dir。

SD2.1 / SDXL 只切换 Model 与 generation; Evaluator 和 scorer 保持一致。

任务 C：生成、评测逻辑与结果保存

10. 生成流程

- Loader 将英文 Prompt 写入每个样本的 GenerationConfig。
- SD2.1 使用 StableDiffusionPipeline，生成 768×768 图片。
- SDXL 使用基础模型 Pipeline，生成 1024×1024 图片。
- 两组实验均为 50 steps、CFG 9、seed 42、1 image / prompt。
- 生成完卸载 T2I 模型，再加载大型 CLIP-FlanT5 scorer。

生成模型变化不会修改 GenAI-Bench evaluator。

11. VQAScore 评测

- 展平所有图片和对应英文 Prompt，并记录 owner index。
- 临时保存 PNG，按 batch 16 输入 CLIP-FlanT5。
- scorer 返回每个 image-prompt pair 的连续 VQAScore。
- 按 owner 恢复到原样本；多图时取平均得到 Prompt score。
- 为每个 skills 标签写一条 skill_metrics，并额外加入 all。

本次每个 Prompt 只有一张图，因此 Prompt score 等于该图 VQAScore。

12. 聚合与结果保存

- aggregate_metric_from_list 按 skill 分组平均，生成全部 task_scores。
- Basic / Advanced 和 10 个细分能力均来自同一套 skill 标签。
- Overall 对有 skills 的 Prompt score 直接取平均。
- Runner 保存模型、generation config、完整分类指标和 error。

results_genaibench_diffusers.json
task_scores / score

任务 C：SD2.1 全部分类指标复现

	Basic	Advanced	Overall
论文 SD2.1	0.750000	0.600000	0.670000
本项目复现	0.752781	0.599709	0.669723
相对误差	+0.37%	-0.05%	-0.04%

	Attribute	Scene	Spatial	Action	Part	Counting	Comparison	Differ.	Negation	Universal
本项目复现	0.7551	0.7726	0.7333	0.7283	0.7041	0.6671	0.6787	0.6540	0.5225	0.6395

复现配置

SD2.1, 768² | 50 steps | CFG 9.0
seed 42 | clip-flant5-xxl

结果判断

Overall 与论文报告基本一致，完成 SD2.1 指标复现。

任务 C: SDXL 全部分类指标复现

	Basic	Advanced	Overall
论文 SDXL	0.8200	0.6300	0.7200
本项目复现	0.8260	0.6375	0.7237
相对误差	+0.73%	+1.19%	+0.51%

	Attribute	Scene	Spatial	Action	Part	Counting	Comparison	Differ.	Negation	Universal
本项目复现	0.8371	0.8511	0.8118	0.8142	0.8180	0.7285	0.7044	0.6963	0.5151	0.6682

复现配置

SDXL-base-1.0, 1024² | 50 steps | CFG 9.0
seed 42 | clip-flant5-xxl

结果判断

Overall 与论文报告基本一致，完成 SDXL 指标复现。

任务 A–C：复现结果总览

Benchmark	测试模型	论文参考	本项目复现	相对误差
GenEval	SD1.5	0.43	0.4208	−2.14%
ELLA / DPG-Bench	SD1.5	63.18	63.68	+0.79%
GenAI-Bench	SD2.1	0.6700	0.669723	−0.04%
GenAI-Bench	SDXL	0.7200	0.7237	+0.51%

任务 D: A-C 接入后, 真正需要解决什么?

问题 1: 数据靠位置约定

- 原流程在阶段间传递 (GenerationConfig, metadata) tuple。
- Prompt、样本编号、图片和分数没有统一的命名字段。
- ELLA 的 questions / dependency 与 GenAI 的 skills 都继续塞入 metadata。

后果: 新增 Benchmark 时容易依赖 tuple 位置和隐式字段, 阶段边界难检查。

问题 2: 模型差异向外泄漏

- 原 adapter 固定传一组 Diffusers 参数, 并假设输出一定有 .images。
- SD、SDXL 还能共用; 新 Pipeline 可能不接受相同参数。
- 模型特有参数若加入公共 Schema, 会不断扩大核心接口。

后果: 接入新模型可能反过来修改 evaluator 或公共 GenerationConfig。

问题 3: 一次运行难以复查

- 输出重点是最终 JSON, 逐样本依据和运行环境没有形成稳定目录。
- YAML 与 CLI 都能给参数, 但覆盖顺序和作用域不够清楚。
- 失败后只能看到短错误, 难判断是数据、生成还是评分阶段。

后果: 同一个分数难以回答“由哪套配置和哪些样本得到”。

任务 D：统一各阶段传递的样本格式

修改前：阶段之间传匿名 tuple

`(GenerationConfig, metadata)`

- Loader 决定 tuple 中放什么。
- SimpleEvaluator 生成后再用 zip 拼回 metadata。
- Postprocessor / Aggregator 只能约定某个 key 在 metadata 中存在。

问题：接口“能传”，但不能说明每个字段的职责。



修改后：两个有名字的对象贯穿流程

生成前：BenchmarkSample

- sample_id / prompt / generation_config
- metadata / references

生成后：SampleEvaluation

- generation / image_paths / scores
- metadata / error

结果：Loader、评分和聚合共享同一份样本身份。

SimpleEvaluator 只编排 load → preprocess → generate → postprocess → aggregate；新增 Benchmark 只实现各阶段的差异。

任务 D：把模型差异收回 Adapter 内部

实际差异	原实现风险	Adapter 中的处理
调用参数不同	所有 Pipeline 都收到固定参数，新增参数又要修改公共 Schema	公共参数保持严格；模型专用值放入 <code>extra_kwargs</code>
支持参数不同	Pipeline 不接受某个参数时，到运行阶段才报错	读取 <code>pipeline.__call__</code> 签名，只保留其支持的参数
输出形状不同	直接访问 <code>output.images</code> ，Adapter 假设过强	统一归一化 <code>.images</code> 、 <code>dict</code> 、 <code>list</code> 和 <code>tuple</code> 输出
执行能力不同	所有模型被迫采用同一种批处理实现	<code>BaseModel.generate</code> 为最小契约；模型可覆盖 <code>generate_batch</code>

Evaluator 只依赖稳定部分

GenerationConfig → BaseModel.generate
GenerationResult(images, debug_info)

YAML 表达模型差异

checkpoint / pipeline / dtype
scheduler / LoRA / extra_kwargs

任务 D：让一次运行可以被解释和复查

配置问题如何解决？

defaults → YAML → CLI override

- `model_args`: 模型路径、Pipeline、dtype。
- `generation`: 尺寸、steps、CFG、seed。
- `eval_args`: 数据路径、scorer 与聚合策略。
- `output`: 目录、图片保存、resume、结构化产物。

收益：同一 Runner 不需要识别 ELLA 或 GenAI-Bench 的专有字段。

结果问题如何解决？

```
runs/<config-fingerprint>/  
config.json  
environment.json  
metrics.json  
samples.jsonl  
errors.jsonl / complete.marker  
images/
```

配置指纹隔离不同实验；逐样本记录连接 Prompt、图片和分数；环境文件记录依赖、GPU 与 commit。

兼容策略：继续输出 `results_<benchmark>_<model>.json`，同时新增 `run artifacts`；已有命令和验收文件不失效。

任务 D：验证重构是否成立

验证层次	要防止的问题	本项目证据
数据与注册	新对象转换时丢 Prompt、skills、questions 或类别	Loader / row converter / evaluator registration 测试
评分语义	公共流程改造后改变 dependency、last-image 或 skill 聚合	ELLA 依赖传播、GenAI scorer batching 与聚合单元测试
真实运行	测试替身可通过，但模型、scorer 或落盘链路不能工作	smoke 验证生成、卸载模型、评分和保存的完整生命周期
论文复现	工程重构没有报错，但最终统计口径发生漂移	固定 checkpoint 与生成参数，保留分类指标并对照官方 report

自动测试

12 passed
ELLA + GenAI 核心语义

规模核对

1,065 / 1,600
ELLA / GenAI Prompts

验收标准

接近论文
A-C 四组 Overall

任务 E：从官方独立脚本到框架内 Evaluator

1. 先拆解官方流程

- 12 个 JSON 分别描述 4 个 Composition 与 8 个 Reasoning 维度。
- `sample.py` 单独生图，并按 `model / dimension` 保存 PNG。
- `evaluate.py` 读取图片，对每条 checklist 做 Yes/No 问答。
- 每张图取有效问题均值，再得到各维度 `mean_score`。

保留：官方数据、问题模板、二值解析和逐图评分定义。

2. 再确定接入边界

- 生图和评分不再由两个外部脚本分别启动。
- Loader 把官方记录转换为 `BenchmarkSample`。
- Postprocessor 把图片展开为 `image-question` 请求。
- Aggregator 输出维度、Composition、Reasoning 与 overall。
- 通过 `@register_evaluator` 接入统一 CLI。

目标：不是包装官方命令，而是成为 T2IEval 内部 evaluator。

3. 选择本地评测方案

- 主论文 evaluator 是 Gemini 2.5 Flash，需要外部 API。
- 当前官方仓库已加入 Qwen3.5，并给出对应评测结果。
- 本项目选择 Qwen3.5-9B：单卡可部署，且属于官方支持分支。
- vLLM 放在独立环境，避免与主项目生成/scorer 依赖冲突。

生成模型：使用 Qwen-Image 验证新模型接入能力。

任务 E：框架内代码如何实现？

新增 evaluator 模块

- `__init__.py`: 导出并触发 evaluator 注册。
- `loader.py`: 解析 12 维官方 JSON、选择维度和限制 Prompt 数。
- `evaluator.py`: 配置、Postprocessor、聚合器与两种运行模式。
- `scorer.py`: Judge Protocol、缓存键、子进程调用和二值解析。
- `qwen_worker.py`: 隔离加载 vLLM / Qwen3.5 并批量推理。

`src/t2i_eval/eval/t2i_corebench/`

补充通用能力

- `PrecomputedImageModel`: 把已有图片包装为标准生成结果。
- `extra_kwargs`: 传递 Qwen-Image 的 `true_cfg_scale`。
- 样本级 `complete.marker`: 长时间生图可断点续跑。
- `ArtifactWriter`: 统一保存样本、问题、配置、环境和指标。
- 两份 YAML: 直接生成与读取已有图片。

原因：这些能力不只服务 CoReBench，其他 evaluator 也可复用。

Evaluator 装配过程

- 注册名称 `t2i_corebench`，由 Registry 提供统一入口。
- `T2ICoreBenchConfig` 校验维度、数据、图片与 Judge 参数。
- `run_mode=generate_only` 时只生图和保存。
- 正常模式装配本地 Qwen Postprocessor。
- 最后绑定 `aggregate_corebench_results`。

```
t2i-eval -e t2i_corebench
-f examples/
run_t2i_corebench_qwen_image.yaml
```

任务 E：数据读取、样本组织与参数配置

数据读取

- 支持本地官方 `data/*.json`，也可从 HF Dataset snapshot 读取。
- 12 个维度名称在 Loader 中集中校验，拒绝未知维度。
- 每条记录读取 ID、Prompt、Checklist、Main Class、Sub Class 和 Remark。
- `dimensions / num_prompts` 支持完整、部分维度和小规模实验。

输出： 所有来源统一转换为 `BenchmarkSample`。

Prompt 与 Checklist 组织

- 原始 Prompt 直接用于 T2I 生成，不做改写。
- Checklist 保存在 metadata，每项保留 question 与 tags。
- 每条 Prompt 按图片数展开为独立 sample，建立稳定 `sample_id`。
- seed 随图片索引递增，使 resume 后仍能定位同一生成请求。

评分单位： 一张图中的每个 checklist question。

当前扩展实验配置

- 生成模型：本地 Qwen-Image, bfloat16。
- 生成参数：1328², 50 steps, True CFG 4.0, seed 0。
- 维度：C-MI、C-MR、R-LR、R-CR，各 90 条 Prompt。
- 资源折中：每条 Prompt 1 张，共 360 张图片。
- Judge：本地 Qwen3.5-9B, vLLM batch 64。

状态： 结果仍在运行，本页只记录实验范围，不报告分数。

任务 E：生成、读取已有图片与本地评分

路径 A：框架内批量生成

- Loader 产生 Qwen-Image 的标准生成配置。
- 公共 Diffusers adapter 逐样本生成并立即落盘。
- 每个样本写入图片、metadata 和完成标记。
- 全部图片完成后卸载 Qwen-Image，释放显存再加载 Judge。

支持：批量运行、进度显示、样本级 resume。

路径 B：读取已有图片

- `image_dir` 指向 model-level 图片目录。
- Loader 按 `dimension/prompt-id-index.png` 查找图片。
- `strict_images=true` 拒绝数量不足的 Prompt 组。
- `PrecomputedImageModel` 将文件读取为 `GenerationResult`。
- 后续评分和聚合与现场生成完全共用。

意义：官方图片或其他 T2I 模型结果无需重新生成。

统一的评分与输出

- 每个 image-question 构造独立请求，Qwen 仅返回 yes / no。
- 无法解析的回答记为 invalid，并单独统计有效回答率。
- `question` → `image` → `dimension` → `Composition` / `Reasoning` / `overall`。
- 输出 `metrics.json` / `samples.jsonl` / `questions.jsonl` 与图片、配置、环境、缓存。

部分评测：`is_partial_evaluation=true`，避免误当完整榜单分数。

任务 E：哪些组件复用，哪些必须新增？

能力	直接复用	CoReBench 新增部分
启动与配置	CLI、YAML、Registry、Runner	注册名与 T2ICoreBenchConfig
数据与生命周期	BenchmarkSample、SimpleEvaluator	12 维 JSON Loader、图片展开规则
图片来源	DiffusersModel、统一 GenerationResult	Qwen-Image YAML;通用 PrecomputedImageModel
单样本评分	Postprocessor 接口、模型卸载与图片路径	Checklist 请求构造、Qwen Judge adapter、二值解析
汇总与保存	ArtifactWriter、samples / config / environment	维度、Composition / Reasoning、valid / cached rate、questions

为什么复用？

启动、生图、样本身份和结果落盘与评分语义无关，不应再实现一套。

为什么新增？

12 维数据、Checklist、Judge Prompt 和聚合层级属于 Benchmark 定义。

任务 E：与官方评测的差异及本地结果局限

对比项	官方方式	本项目本地方式
Judge	主论文使用 Gemini 2.5 Flash；当前仓库也支持 Qwen 系列	使用官方支持的 Qwen3.5-9B, 本地 vLLM 子进程, 无外部 API
Prompt / 解析	官方 system template, temperature 0, Yes/No, 多轮重试	移植同一 template、二值规则、temperature 0、repetition penalty 与重试策略
执行入口	sample.py 与 evaluate.py 分开运行	一个 evaluator 支持现场生成、generate-only 和已有图片评测
输出口径	每个模型、每个维度各写一个 JSON / mean score	统一保存问题、图片、维度及 Composition / Reasoning / overall
随机与规模	默认 12 维、4 images / Prompt, 进程级随机状态	当前 4 维、1 image / Prompt；显式逐图 seed, 便于 resume

Judge 局限

Qwen 与 Gemini 对小物体、文字、否定和复杂关系可能给出不同判断。

采样局限

单图无法估计四图方差；部分维度 overall 不能与完整榜单直接比较。

环境局限

Qwen / vLLM 版本、精度、invalid rate 和生成 seed 都会影响最终结果。

Q & A

感谢聆听